ADDING CART AND CHECKOUT TO YOUR EXPERIENCE

Table of Contents

ADDING CART AND CHECKOUT TO YOUR EXPERIENCE

Last Updated: 9/12/2019

Manage the Cart and Checkout process for the consumer.

TIPS:

- Before using this guide, read **Using Nike APIs (https://developer.niketech.com/nde-docs/doc/getting-started/using-nike-apis.html)** and **Cart & Checkout Overview (https://developer.niketech.com/nde-docs/doc/commerce/checkout/overview-checkout.html)**.
- Use this Developer's Guide as a supplement to the API Reference for detailed use cases. See **API Quick Reference** for links to all the API Reference docs discussed in this guide.
- The steps involving **Payment** are covered in **Adding Payment to Your Experience (https://developer.niketech.com/nde-docs/doc/commerce/payment/use-payment.html)**

Introduction

What is a Cart?

In e-commerce, the shopping cart (also called basket or bag) allows consumers to collect and compare products that they are considering for purchase, but without requiring them to enter their shipping and billing information.

At Nike, a cart contains the following:

- Products & services with respective prices, discounts, and quantities
- Promotion codes
- Totals

What is a Checkout?

A checkout is different from a cart in that it represents a consumer's intent to complete a purchase and must include additional information necessary for the fulfillment of an order.

At Nike, a checkout includes the **info from the cart plus the following**:

- Shipping method(s)
- Shipping address(es)
- Payment method(s)
- Billing address(es)
- Taxes

Thus, both the cart and the checkout serve a specific purpose within the shopping flow, based on the level of **commitment to purchase** that the consumer has at a particular moment.

What is a Wish List?

Sometimes consumers are not ready to purchase items or services and want to set them aside to buy at a later date. Consumers can do this by creating a Wish List. Consumers can create an unlimited number of Wish Lists and can add an unlimited number of items and services. When consumers are ready to purchase an item from their Wish List, they use the app/experience to add the item to their Cart and remove the item from their Wish List. While consumers can also manage their future purchases using a Cart, Wish Lists allow greater flexability by allowing multiple Wish Lists that they can name.

Key Terms

Term	Definition
Cart	The virtual shopping cart used by Nike consumers to collect and compare items for purchase
Checkout	A collection of data describing what may become a consumer order
Fapiao	Tax-related invoice offered to China customers only

Cart

✔ **Manage a consumer's shopping cart and get product pricing**

✔ **Check if a product can be purchased or not**

✔ **Summarize a cart prior to checkout**

Now that you know what a cart is, let's explore how to add it to your experience.

Step 1: Create the Cart

The first step in managing a consumer's cart is to create the cart using the **Carts API (https://developer.niketech.com/docs/projects/Carts%20V2?tab=api)**. For example, this could be done when the consumer chooses to add their first product(s) to the cart.

To create the cart, execute a request to the **Create or Update a Cart by Cart ID (https://developer.niketech.com/docs/projects/Carts%20V2?tab=api#cart-operations-create-or-update-a-cart-by-cart-id-put)** or **Create or Update a Cart by Filter Criteria (https://developer.niketech.com/docs/projects/Carts%20V2?tab=api#cart-operations-create-or-update-a-cart-by-filter-criteria-put)** endpoint.

> **TIP:** A cart is owned by one consumer (member, guest, or employee) who must be authenticated. If an attempt is made to manage a cart when no, or incorrect, authentication is provided, an error will be returned by the Carts API.

Sample **Create or Update a Cart by Cart ID (https://developer.niketech.com/docs/projects/Carts%20V2?tab=api#cart-operations-create-or-update-a-cart-by-cart-id-put)** request URI:

```
https://api.nike.com/buy/carts/v2/61bc115b-16e5-43b5-bcaf-dd6168c543f8
```

If you successfully create the cart, you will get product pricing and, if the consumer is a member who has saved a shipping address, you will get their default shipping address and recipient (i.e. contact) info from their Nike profile.

Step 2: Get a Cart

Now that the cart has been created, you can display the cart to the consumer, for example if they continue shopping and then later want to see the cart details again.

To get a cart, you have a few options depending on what information you need:

1. Execute a request to the **Get a Cart by Cart ID (https://developer.niketech.com/docs/projects/Carts%20V2?tab=api#cart-operations-get-a-cart-by-cart-id-get)**, **Get a Cart by Filter Criteria (Query Param) (https://developer.niketech.com/docs/projects/Carts%20V2?tab=api#cart-operations-get-a-cart-by-filter-query-param-get-2)**, or **Get a Cart by Path Parameters (https://developer.niketech.com/docs/projects/Carts%20V2?tab=api#cart-operations-get-a-cart-by-path-parameters-get-1)** endpoint of

the Carts API to get the full details of the cart.

2. Execute a request to the **Get a Cart Summary by Cart ID (https://developer.niketech.com/docs/projects/Carts%20V2?tab=api#cart-operations-get-a-cart-summary-by-cart-id-get-1)** endpoint, only if you support PayPal Express payment type and want to display promo codes on the order confirmation.

TIP: For more info on how to use the `?filter` query parameter, see **Using Nike APIs (https://developer.niketech.com/nde-docs/doc/getting-started/using-nike-apis.html#query-parameters)**.

Step 3: Modify the Cart

To add or remove products, services, and promotion codes from a cart, execute a request to the **Modify a Cart by Cart ID (https://developer.niketech.com/docs/projects/Carts%20V2?tab=api#cart-operations-modify-a-cart-by-cart-id-patch-1)** or **Modify a Cart by Filter Criteria (https://developer.niketech.com/docs/projects/Carts%20V2?tab=api#cart-operations-modify-a-cart-by-filter-criteria-patch-1)** endpoint.

TIP: Prices and subtotals are recalculated and returned in the response to each request.

To delete **all** of the products in the cart, execute a request to **Delete All Items from a Cart by Cart ID (https://developer.niketech.com/docs/projects/Carts%20V2?tab=api#cart-operations-delete-all-items-from-a-cart-by-cart-id-delete-1)** or **Delete All Items from a Cart by Filter Criteria (https://developer.niketech.com/docs/projects/Carts%20V2?tab=api#cart-operations-delete-all-items-from-a-cart-by-filter-criteria-delete-1)** endpoints.

The delete operation is optional, even if the cart is empty; member's carts will automatically purge from storage after 90 days of inactivity, while guest carts will purge at 30 days.

Step 4: Get a Cart Summary

The consumer has finished add products to the cart, and they might wish to see a summary before proceeding to checkout. Use the **Cart Reviews API (https://developer.niketech.com/docs/projects/Cart%20Reviews?tab=api)** to enhance a cart summary with taxes, estimated delivery date(s), shipping group(s) (when applicable), and updated subtotals.

To get a cart summary, execute a request to the **Augment a Cart (**

https://developer.niketech.com/docs/projects/Cart%20Reviews?tab=api#cart-reviews-augment-a-cart-post) endpoint with a complete cart.

> **NOTE**: It is not required to create a cart with the Carts API prior to sending a request to the Cart Reviews API. Instead of using Cart ID in the request, send the **country**, **currency**, and **brand** associated with the consumer.

Depending on what additional info you include in the request, you can get additional info in the response, as follows:

- To get sales tax and shipping tax, the request must include postal code.
- To get estimated delivery date(s), the request must include the shipping method(s).
- To get shipping group information, the request must include the shipping method and the shipping address associated with each product.

Sample **Augment a Cart (https://developer.niketech.com/docs/projects/Cart%20Reviews?tab=api#cart-reviews-augment-a-cart-post)** request URI:

```
https://api.nike.com/buy/cart_reviews/v1/
```

> **TIPS:**
>
> - Shipping group refers to the grouping of products into multiple shipments with potentially different delivery dates. This is done automatically for you based on Nike business rules.
> - For China consumers, you can capture and include **Fapiao invoice (https://www.sirva.com/docs/default-source/default-document-library/what-are-fapiaos-and-why-do-they-matter-.pdf)** info in the request and it will be returned in the response.

Shipping Options

✔ **Get available shipping methods and estimated delivery dates**

Now that your consumer has finalized their cart, it's time to begin the checkout process. The first step is for them to select a shipping method.

Consumers are accustomed to selecting a shipping method (e.g. Standard, Two-Day, Next-Day) during the checkout process. But, how do you know which methods to present to them, based on

their shopping context?

Use the **Shipping Options API (https://developer.niketech.com/docs/projects/Shipping%20Options?tab=api#shipping-options-post)** to retrieve the available shipping methods for a consumer's checkout, including any associated costs, estimated delivery dates, or discounts (such as free shipping for members).

To get the shipping options, execute a request to the Shipping Options endpoint.

Sample **Shipping Options API (https://developer.niketech.com/docs/projects/Shipping%20Options?tab=api#shipping-options-post)** request URI:

```
https://api.nike.com/buy/shipping_options/v2
```

TIP: Although optional, including a shippingAddress is recommended whenever possible. In China, shipping methods can vary based on the province, city, and district combination. Also, for certain countries (e.g. US), including the shipping address can get you an estimated delivery date versus an estimated delivery range.

Shipping Address Validation

✔ **Ensure the shipping address is deliverable**

After the consumer selects or provides a shipping address, validate the address with the **Address Validator API (https://developer.niketech.com/docs/projects/AddressValidator?tab=api)**. This endpoint can validate any type of address, e.g. billing address.

This service calls a third party vendor to validate the shipping address passed in the request against an address database. The service response contains a `verficationCode`, `score`, and an address. Based on the quality of the address match, the service returns either the consumer-provided address or a corrected address. See the table below to understand how the verificationCode and score work together to determine what actions the consumer needs to take next.

Verification Code	Score	Consumer needs to
VERIFIED	95 or	Address provided by the consumer is verified and is returned in the

Verification Code	Score	Consumer needs to
	greater	response. No further action is required by the consumer.
VERIFIED	less than 95	Address provided by the consumer is missing information and a recommended address is returned in the response. Consumer needs to review the recommended address and decide to accept it or retry with a different address.
PARTIALLY_VERIFIED, UNVERIFIED, AMBIGUOUS, CONFLICT, REVERTED	any	Address provided by the consumer could not be verified and is returned in the response. Consumer needs to correct the address and retry.

TIP: The third party address validation service has a 512 character limit restriction on each address field and a 1024 character limit for the entire address. The client should truncate characters in any address field exceeding the field limit or address limit before making the request.

See the **Address Validation Service (https://confluence.nike.com/pages/viewpage.action?pageId=270586569)** page for more information on request and response field mappings between the Address Validator API and the third party.

Sample **Address Validator API (https://developer.niketech.com/docs/projects/AddressValidator?tab=api)** POST request URI. This is a synchronous service and is not JWT-protected:

```
https://api.nike.com/location/address_validator/v1
```

Previewing a Checkout

✔ **Validate a checkout for fulfillment**

Next, let's make sure that the checkout details are accurate and that the process can proceed to the payment steps.

Can I Skip This?

It is not required to execute a request to **Request a Checkout Preview (https://developer.niketech.com/docs/projects/Checkouts%20V2?tab=api#checkout-preview-request-checkout-preview-put)** in order for the consumer to complete their purchase. However, it is recommended to increase the chance of a successful checkout.

You can use the info in the response to display the final payment amount to the consumer. Once the consumer confirms the payment method details and places the order, there will be a better chance of success.

NOTE: Checkout Preview (and Checkout Submit in the next steps) operates asynchronously. This means that after you execute the initial request, you call another endpoint to get the result. See **Using Nike APIs (https://developer.niketech.com/nde-docs/doc/getting-started/using-nike-apis.html#asynchronous-operation)** for more details.

Step 1: Request Checkout Preview

Execute a request to the **Request a Checkout Preview (https://developer.niketech.com/docs/projects/Checkouts%20V2?tab=api#checkout-preview-request-checkout-preview-put)** endpoint.

The API ensures that the products, shipping method(s), and shipping address(es) are valid based on Nike pricing and address rules. You can also get product pricing, sales tax, shipping fee and tax, estimated delivery date(s), and checkout subtotals in the response.

TIP: For more context, see a step-by-step example of all the requests in a checkout in the diagram in the **Best Practices** section of this document. For more info about Payment, see **Adding Payment to Your Experience (https://developer.niketech.com/nde-docs/doc/commerce/payment/use-payment.html)**.

Sample **Request a Checkout Preview (https://developer.niketech.com/docs/projects/Checkouts%20V2?tab=api#checkout-preview-request-checkout-preview-put)** request URI:

```
https://api.nike.com/buy/checkout_previews/v2/61bc115b-16e5-43b5-bcaf-dd6168c543f8
```

Step 2: Retrieve Checkout Preview Job

After calling **Request a Checkout Preview (https://developer.niketech.com/docs/projects/Checkouts%20V2?tab=api#checkout-preview-request-checkout-preview-put)** and receiving a HTTP 202 response, execute a request to **Retrieve Checkout Preview Job (https://developer.niketech.com/docs/projects/Checkouts%20V2?tab=api#checkout-preview-retrieve-checkout-preview-job-get-1)** using the same checkout ID to check the status of your job.

To know if the job is done, check the value of the **status** field in the response body as follows:

- `"status": "PENDING"`: job processing has not started
- `"status": "IN_PROGRESS"`: job processing is in progress
- `"status": "COMPLETED"`: job has completed

Once you receive a job status of COMPLETED, get the results of your job by parsing the data in the **response** object.

Sample **Retrieve Checkout Preview Job (https://developer.niketech.com/docs/projects/Checkouts%20V2?tab=api#checkout-preview-retrieve-checkout-preview-job-get-1)** request URI:

```
https://api.nike.com/buy/checkout_previews/v2/jobs/
61bc115b-16e5-43b5-bcaf-dd6168c543f8
```

Submitting a Checkout

✔ **Submit a checkout for fulfillment**

Step 1: Request Checkout Submit

Execute a request to the **Request Checkout Submit (https://developer.niketech.com/docs/projects/Checkouts%20V2?tab=api#checkout-request-a-checkout-submit-put-1)** endpoint when your consumer is ready to complete their purchase.

The API performs the final validations of the consumer's information, requests payment authorization, and if everything succeeds, submits the checkout for fulfillment.

TIPS:

- You must have previously called the Payment Preview API to collect the required payment information, most notably the mandatory Payment Preview **id**. See the **Adding Payment to Your Experience (https://developer.niketech.com/nde-docs/doc/commerce/payment/use-payment.html)** for more info.
- Optionally, for Japan only, send `GIFT_RECEIPT` in the **invoiceInfo** block, which prevents prices from being printed on the packing slip that is included with the product shipment.
- Optionally, for China only, send `ELECTRONIC_FAPIAO` in **invoiceInfo** for Fapiao, which is a special tax invoice. If the consumer indicates a preference for Fapiao, they can enter a personal message to be used as a title for the invoice. For example:

```
"invoiceInfo": {
    "type": "ELECTRONIC_FAPIAO",
    "detail": "The consumer's personal title for the invoice"
}
```

Sample **Request Checkout Submit (https://developer.niketech.com/docs/projects/Checkouts%20V2?tab=api#checkout-request-a-checkout-submit-put-1)** request URI:

```
https://api.nike.com/buy/checkouts/v2/61bc115b-16e5-43b5-bcaf-
dd6168c543f8
```

Step 2: Retrieve Checkout Submit Job

After calling the **Request Checkout Submit (https://developer.niketech.com/docs/projects/Checkouts%20V2?tab=api#checkout-request-a-checkout-submit-put-1)** endpoint and receiving a HTTP 202 response, execute a request to the **Retrieve Checkout Submit Job (https://developer.niketech.com/docs/projects/Checkouts%20V2?tab=api#checkout-retrieve-checkout-submit-job-get-1)** using the same checkout ID to check the status of your job.

The same job statuses apply for this endpoint as they do for Retrieve Checkout Preview Job. Once you observe a job status of COMPLETED, get the results of your job by parsing the data in the **response** object.

Sample **Retrieve Checkout Submit Job (https://developer.niketech.com/docs/projects/Checkouts%20V2?tab=api#checkout-retrieve-checkout-submit-job-get-1)** request URI:

```
https://api.nike.com/buy/checkouts/v2/jobs/61bc115b-16e5-43b5-bcaf-
dd6168c543f8
```

Wish Lists

✔ **Manage a consumer's Wish Lists (member/employee only) of products and services**

Congratulations, you have just submitted your first checkout for fulfillment! But wait, there's one more OPTIONAL step for you to know about, managing a Nike member/employee's Wish Lists using the **Wish Lists API (https://developer.niketech.com/docs/projects/Wishlist?tab=api)**.

Features:

- Store unlimited Wish Lists per consumer that consumers can name.
- Get product pricing and availability for products added to the list.
- Member and employee support only. **Guest consumers (non-members) cannot save Wish Lists**.

Step 1: Create a Wish List

Execute a request to the **Create or Update a List (https://developer.niketech.com/docs/projects/Wishlist?tab=api#list-operations-create-or-update-a-list-put)** endpoint to create an empty Wish List. **Step 2** explains how to add items to a Wish List.

- `name` must be unique for a user/country combination.
- You can only set a Wish List as the default for a country when creating it. You cannot update it as the default later.
- When you create a new Wish List with `isDefault` set to true, the `isDefault` field will be set to false for the rest of a consumer's Wish Lists for the same country.
- If the `id` path parameter does not match the `id` of the Wish List, a new Wish List will be created.

TIP: You generate the unique Wish List `id` in UUID format and pass it as a path parameter.

Listed below is a sample **Create or Update a List (https://developer.niketech.com/docs/projects/Wishlist?tab=api#list-operations-create-or-update-a-list-put)** PUT request URI that creates a new Wish List with `id` 93a333a2-907b-46f1-b9ac-469489909057. This endpoint is not JWT-restricted.

```
https://api.nike.com/buy/lists/v1/93a333a2-907b-46f1-b9ac-469489909057
```

Step 2: Modify a Wish List

Now that you've created a Wish List for the consumer, allow them to add or remove items, delete the list, or rename it.

Add Item to List

To add a product or service to an existing Wish List, execute a request to the **Add Item to List (https://developer.niketech.com/docs/projects/Wishlist?tab=api#list-item-operations-add-item-to-list-put)** endpoint.

- Current product pricing is provided in the response.
- If you add a product that is already on the list, the product will be replaced.
- Only the product being added is included in the response, not all products in the list.
- The Wish List item `country` will override the Wish List `country` when looking up prices.
- Because this endpoint can replace items, the response includes `isCurrentPriceChanged` set to true when an item's currentPrice has changed since creation.
- The response contains an `isAvailable` flag indicating whether or not the product is purchasable. This value is determined by the **Availability service (https://developer.niketech.com/docs/projects/Availability?tab=api)**.

TIP: You generate the unique Wish List item `id` in UUID format and pass it in the path parameter.

Listed below is a sample **Add Item to List (https://developer.niketech.com/docs/projects/Wishlist?tab=api#list-item-operations-add-item-to-list-put)** PUT request URI that creates a new Wish List item with `id` 93a333a2-907b-46f1-b9ac-469489909123. This endpoint is not JWT-restricted.

```
https://api.nike.com/buy/list_items/v1/
93a333a2-907b-46f1-b9ac-469489909123
```

Add Multiple Items to List

To add more than one item at a time to one or more Wish Lists, execute a request to the **Add Item to List (https://developer.niketech.com/docs/projects/Wishlist?tab=api#list-item-operations-**

add-item-to-list-patch) endpoint.

- Only adding new items is supported, not replacing existing items.
- Current product pricing is provided in the response.
- The Wish List item `country` will override the Wish List `country` when looking up prices.
- You can add items to different Wish Lists in the same call by supplying different `wishlistId`s in each value object in the PATCH request body.
- Unless you supply filter query parameters to restrict the result set, all newly added items are returned in the response.
- The response contains an `isAvailable` flag indicating whether or not the product is purchasable. This value is determined by the **Availability service (https://developer.niketech.com/docs/projects/Availability?tab=api)**.

TIP: You generate each unique Wish List item `id` in UUID format and pass them in the PATCH request body.

Listed below is a sample **Add Item to List (https://developer.niketech.com/docs/projects/Wishlist?tab=api#list-item-operations-add-item-to-list-patch)** PATCH request limiting the response to include only the added items from Wish Lists with 93a333a2-907b-46f1-b9ac-469489909057 and 93a333a2-907b-46f1-b9ac-469489909000 ids. This endpoint is not JWT-restricted.

```
http://api.nike.com/buy/list_items/
v1?filter=wishlistId(93a333a2-907b-46f1-b9ac-469489909057,93a333a2-907b
-46f1-b9ac-469489909000)
```

Remove Item from List

To delete a single item from a list, execute a request to the **Remove Item from List (https://developer.niketech.com/docs/projects/Wishlist?tab=api#list-item-operations-remove-item-from-list-delete)** endpoint passing the Wish List item `id` as a path parameter.

TIP: You do not need to send the `wishlistId` in the DELETE request. You only need to send the Wish List item `id` you want to delete in the path parameter.

Listed below is a sample **Remove Item from List (https://developer.niketech.com/docs/projects/Wishlist?tab=api#list-item-operations-remove-item-from-list-delete)** DELETE request URI to delete Wish List item with `id` 93a333a2-907b-46f1-b9ac-469489909123. This

endpoint is not JWT-restricted.

```
https://api.nike.com/buy/list_items/v1/
93a333a2-907b-46f1-b9ac-469489909123
```

Remove Multiple Items from List

To delete several items from a Wish List, execute a request to the **Remove Items from List (https://developer.niketech.com/docs/projects/Wishlist?tab=api#list-item-operations-remove-items-from-list-delete)** endpoint passing the list of Wish List item ids as a filter query parameter.

> TIP: You do not need to send the `wishlistId` in the DELETE request. You only need to send the Wish List item `id` you want to delete in the filter query parameter.

Listed below is a sample **Remove Items from List (https://developer.niketech.com/docs/projects/Wishlist?tab=api#list-item-operations-remove-items-from-list-delete)** DELETE request to delete Wish List items with `id` 93a333a2-907b-46f1-b9ac-469489909057 and 93a333a2-907b-46f1-b9ac-469489909000. This endpoint is not JWT-restricted.

```
https://api.nike.com/buy/list_items/
v1?filter=id(93a333a2-907b-46f1-b9ac-469489909057,93a333a2-907b-46f1-b9
ac-469489909000)
```

Delete a List

To delete a Wish List, execute a request to the **Delete a List (https://developer.niketech.com/docs/projects/Wishlist?tab=api#list-operations-delete-a-list-delete)** endpoint passing the Wish List `id` a a path parameter. Note that **all of the items on the list will be removed**.

Listed below is a sample **Delete a List (https://developer.niketech.com/docs/projects/Wishlist?tab=api#list-operations-delete-a-list-delete)** DELETE request URI that deletes Wish List 93a333a2-907b-46f1-b9ac-469489909057 and all of its items. This endpoint is not JWT-restricted.

```
https://api.nike.com/buy/lists/v1/93a333a2-907b-46f1-b9ac-469489909057
```

Change the name of the Wish List

Make a request to the **Create or Update a List (https://developer.niketech.com/docs/projects/Buy%20Lists?tab=api#list-operations-create-or-update-a-list-put)** endpoint to update the name of an existing Wish List.

- Pass the same `country` and `brand` values when the Wish List was created and pass an updated Wish List `name`.
- `name` must be unique for a user/country combination.
- Updating lists is limited to changing the list name only.
- If the `id` path parameter does not match the `id` of the Wish List, a new Wish List will be created.

TIP: Use the same Wish List `id` as the one used to create it and pass it in as a path parameter.

Sample **Create or Update a List (https://developer.niketech.com/docs/projects/Buy%20Lists?tab=api#list-operations-create-or-update-a-list-put)** PUT request URI that updates the name of Wish List with `id` 93a333a2-907b-46f1-b9ac-469489909057. This endpoint is not JWT-restricted.

```
https://api.nike.com/buy/lists/v1/93a333a2-907b-46f1-b9ac-469489909057
```

Step 3: Get a Wish List

Allow the consumer to view the **header info** of all their Wish Lists, or just one.

Retrieve Lists for Authenticated User

To retrieve **header info** for all lists for an authenticated consumer, execute a request to the **Retrieve Lists for Authenticated User (https://developer.niketech.com/docs/projects/Wishlist?tab=api#list-operations-retrieve-lists-for-authenticated-user-get)** endpoint.

- The list items are **not** included in the response. To get the list items, execute a request to the **Retrieve Items by List (https://developer.niketech.com/docs/projects/Wishlist?tab=api#list-item-operations-retrieve-items-by-list-get)** endpoint with the appropriate list identifier.
- You can use filter and field query parameters to restrict the result set.

Listed below is a sample **Retrieve Lists for Authenticated User (**

https://developer.niketech.com/docs/projects/Wishlist?tab=api#list-operations-retrieve-lists-for-authenticated-user-get) GET request URI that retrieves all of a consumer's Wish Lists for the US. This endpoint is not JWT-restricted.

```
https://api.nike.com/buy/lists/v1?filter=country(US)
```

Retrieve a List by ID

To retrieve **header info** for a single list, execute a request to the **Retrieve a List by ID (https://developer.niketech.com/docs/projects/Wishlist?tab=api#list-operations-retrieve-a-list-by-id-get)** endpoint passing the Wish List `id` as a path parameter.

> **TIP:** The list items are **not** included in the response. To get the list items, execute a request to the Retrieve Items by List endpoint with the appropriate list identifier.

Listed below is a sample **Retrieve a List by ID (https://developer.niketech.com/docs/projects/Wishlist?tab=api#list-operations-retrieve-a-list-by-id-get)** GET request URI for Wish List `id` 93a333a2-907b-46f1-b9ac-469489909057. This endpoint is not JWT-restricted.

```
https://api.nike.com/buy/lists/v1/93a333a2-907b-46f1-b9ac-469489909057
```

Retrieve Items by List

To retrieve all items in a consumer's Wish List, execute a request to the **Retrieve Items by List (https://developer.niketech.com/docs/projects/Wishlist?tab=api#list-item-operations-retrieve-items-by-list-get)** endpoint.

You can use filter, anchor, count, fields, and sort query parameters to restrict the result set.

Listed below is a sample **Retrieve Items by List (https://developer.niketech.com/docs/projects/Wishlist?tab=api#list-item-operations-retrieve-items-by-list-get)** GET request URI that returns all items in the Wish List with `id` 3ebf8798-2c86-4e29-a67b-7435ebad62af. This endpoint is not JWT-restricted.

```
https://api.nike.com/buy/list_items/
v1?filter=wishlistId(3ebf8798-2c86-4e29-a67b-7435ebad62af)
```

Retrieve Item by ID

To retrieve a single list item, execute a request to the **Retrieve Item by ID (**

https://developer.niketech.com/docs/projects/Wishlist?tab=api#list-item-operations-retrieve-item-by-id-get) endpoint, passing the client generated Wish List item `id` as a URL parameter.

You can use the fields query parameter to restrict the result set.

Listed below is a sample **Retrieve Item by ID (https://developer.niketech.com/docs/projects/Wishlist?tab=api#list-item-operations-retrieve-item-by-id-get)** GET request URI. This endpoint is not JWT-restricted.

```
https://api.nike.com/buy/list_items/v1/
93a333a2-907b-46f1-b9ac-469489909057
```

Step 4: Purchase from a Wish List

To allow consumers to purchase items on their Wish List, you'll need to

1. Call **Retrieve Lists for Authenticated User (https://developer.niketech.com/docs/projects/Wishlist?tab=api#list-operations-retrieve-lists-for-authenticated-user-get)** to get the Wish List id or call **Retrieve a List by id (https://developer.niketech.com/docs/projects/Wishlist?tab=api#list-operations-retrieve-a-list-by-id-get)** if you already know the Wish List id.
2. Call **Retrieve Items by List (https://developer.niketech.com/docs/projects/Wishlist?tab=api#list-item-operations-retrieve-items-by-list-get)** to get the item id or call **retrieve item by ID (https://developer.niketech.com/docs/projects/Wishlist?tab=api#list-item-operations-retrieve-item-by-id-get)** if you already know the Wish List item id.
3. Using the Wish List data from Step 1 and Wish List item data from Step 2, call **Create or Update a Cart (https://developer.niketech.com/docs/projects/Carts%20V2?tab=api#cart-operations-create-or-update-a-cart-by-cart-id-put)** to add the Wish List item to Cart.
4. Using the Wish List item `id` from Step 2, call **Remove Item from List (https://developer.niketech.com/docs/projects/Wishlist?tab=api#list-item-operations-remove-item-from-list-delete)**.
5. (Optional) If there are no items left in the Wish List, call **Delete a List (https://developer.niketech.com/docs/projects/Wishlist?tab=api#list-operations-delete-a-list-delete)**.

API Quick Reference

Carts

- Create or Update a Cart by Cart ID (https://developer.niketech.com/docs/projects/Carts%20V2?tab=api)
- Modify a Cart by Cart ID (https://developer.niketech.com/docs/projects/Carts%20V2?tab=api)
- Delete All Items from a Cart by Cart ID (https://developer.niketech.com/docs/projects/Carts%20V2?tab=api)
- Get a Cart by Cart ID (https://developer.niketech.com/docs/projects/Carts%20V2?tab=api)
- Get a Cart by Filter Criteria (Query Param) (https://developer.niketech.com/docs/projects/Carts%20V2?tab=api)
- Create or Update a Cart by Filter Criteria (https://developer.niketech.com/docs/projects/Carts%20V2?tab=api)
- Modify a Cart by Filter Criteria (https://developer.niketech.com/docs/projects/Carts%20V2?tab=api)
- Get a Cart by Filter Criteria (Path Param) (https://developer.niketech.com/docs/projects/Carts%20V2?tab=api)
- Delete All Item from a Cart by Filter Criteria (https://developer.niketech.com/docs/projects/Carts%20V2?tab=api)

Shipping Options

- Shipping Options (https://developer.niketech.com/docs/projects/Shipping%20Options?tab=api)

Address Validator

- Address Validator (https://developer.niketech.com/docs/projects/AddressValidator?tab=api)

Cart Reviews

- Augment a Cart (https://developer.niketech.com/docs/projects/Cart%20Reviews?tab=api)

Checkouts

- Request Checkout Preview (https://developer.niketech.com/docs/projects/Checkouts%20V2?tab=api)

- Retrieve Checkout Preview Job (https://developer.niketech.com/docs/projects/Checkouts%20V2?tab=api)
- Retrieve Checkout Preview Results (https://developer.niketech.com/docs/projects/Checkouts%20V2?tab=api)
- Request Checkout Submit (https://developer.niketech.com/docs/projects/Checkouts%20V2?tab=api)
- Retrieve Checkout Submit Job (https://developer.niketech.com/docs/projects/Checkouts%20V2?tab=api)
- Retrieve Checkout Results (https://developer.niketech.com/docs/projects/Checkouts%20V2?tab=api)
- Request Checkout Submit (Launch) (https://developer.niketech.com/docs/projects/Checkouts%20V2?tab=api)

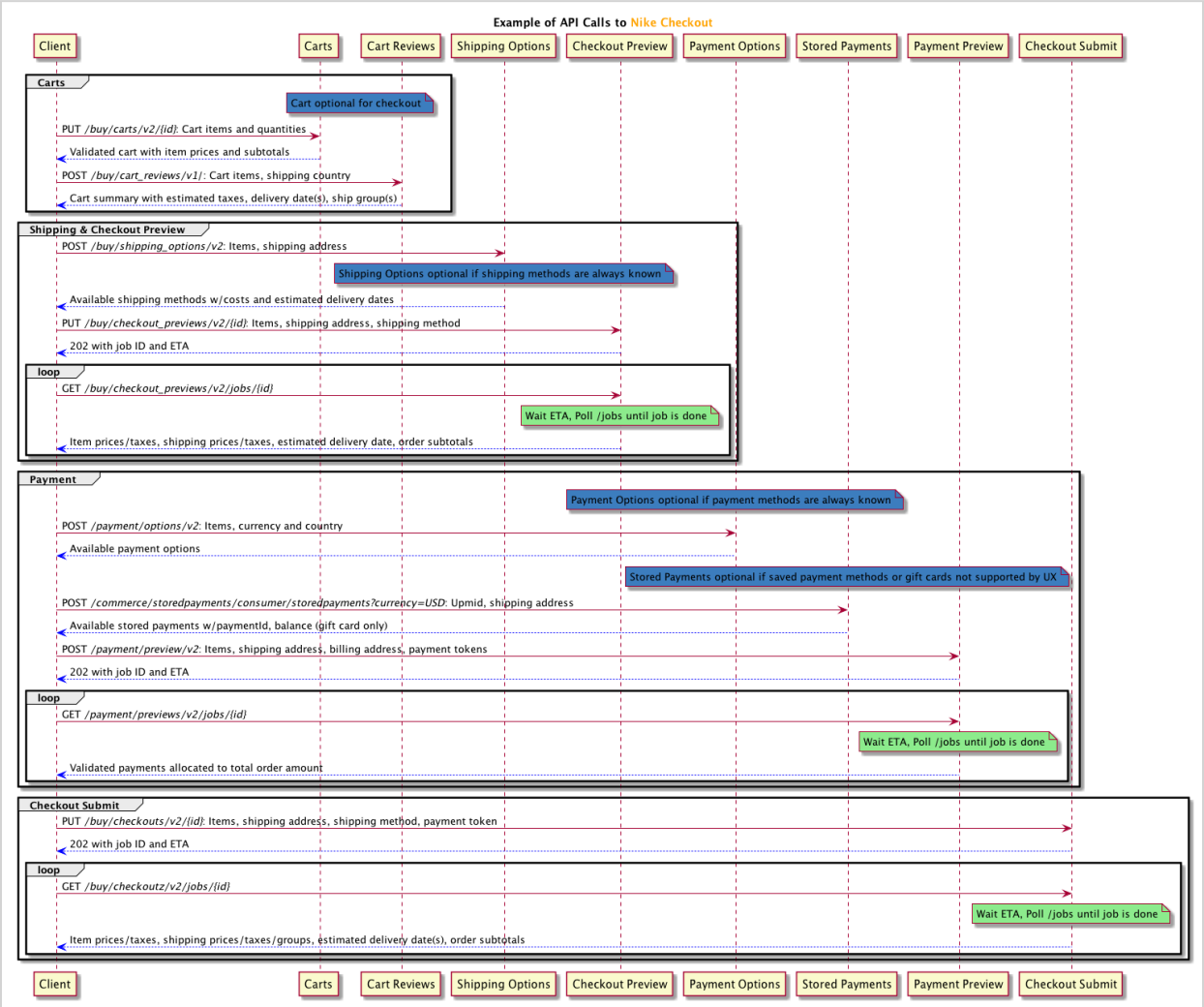
Wish Lists

- Create or Update a List (https://developer.niketech.com/docs/projects/Wishlist?tab=api#list-operations-create-or-update-a-list-put)
- Delete a List (https://developer.niketech.com/docs/projects/Wishlist?tab=api#list-operations-delete-a-list-delete)
- Retrieve a List by ID (https://developer.niketech.com/docs/projects/Wishlist?tab=api#list-operations-retrieve-a-list-by-id-get)
- Retrieve Lists for Authenticated User (https://developer.niketech.com/docs/projects/Wishlist?tab=api#list-operations-retrieve-lists-for-authenticated-user-get)
- Add Item to List (https://developer.niketech.com/docs/projects/Wishlist?tab=api#list-item-operations-add-item-to-list-put)
- Add Item to List (PATCH) (https://developer.niketech.com/docs/projects/Wishlist?tab=api#list-item-operations-add-item-to-list-patch)
- Remove Item from List (https://developer.niketech.com/docs/projects/Wishlist?tab=api#list-item-operations-remove-item-from-list-delete)
- Remove Items from List (https://developer.niketech.com/docs/projects/Wishlist?tab=api#list-item-operations-remove-items-from-list-delete)
- Retrieve Items by List (https://developer.niketech.com/docs/projects/Wishlist?tab=api#list-item-operations-retrieve-items-by-list-get)
- Retrieve Item by ID (https://developer.niketech.com/docs/projects/Wishlist?tab=api#list-item-operations-retrieve-item-by-id-get)

Best Practices

Here are some best practices. We'll start with an example sequence of API calls to execute an entire checkout:

Example Implementation Diagram



User Types

The Cart & Checkout APIs support 3 distinct user types:

- Member: user has logged in with their Nike+ account credentials

- Guest: user has not logged in (anonymous user)
- Employee: user is an employee of Nike or a subsidiary and has logged in with swoosh.com credentials (also known as Swoosh user type)

Depending on user type, certain aspects of the calls that you make to the Cart & Checkout APIs might need to be modified. Also, consider that not all user types might apply to your app (e.g. you might only support Members).

See the User Types section of the **Using Nike APIs (https://developer.niketech.com/nde-docs/doc/getting-started/using-nike-apis.html#user-types)** guide for more information.

Request Headers

The following request headers are common to all of the Cart & Checkout APIs:

Header Name	Description	Member	Guest	Employee
Accept	Content type you will accept in response, application/json is only value allowed	X	X	X
Content-Type	Content type of the request, application/json is only value allowed	X	X	X
Authorization	Your access token in the format of `Bearer {token}` indicating the consumer is logged in	X		X
x-nike-visitorid	Identifier for the guest (i.e. not logged-in) consumer, validated by the Edge router and passed through to the service		X	

TIP: For the Authorization header, use the token for the consumer's login session that you obtained from Nike Unite/Identity, prefixed by **Bearer ** (note the single space after Bearer). This is necessary for Nike to verify that you are authorized to perform the requested operation on behalf of the consumer.

Supported Countries & Currencies

For the list of country code and currency code combinations supported by Cart & Checkout see **Countries & Currencies (https://developer.niketech.com/nde-docs/doc/commerce/**

checkout/checkout-country-currency.html)

Idempotence

Idempotence (http://restcookbook.com/HTTP%20Methods/idempotency/) means that the result of a successful request is independent of the number of times it is executed. What does that mean for the Checkout API? Each PUT request to Request a Checkout Preview (https://developer.niketech.com/docs/projects/Checkouts%20V2?tab=api#checkout-preview-request-checkout-preview-put) and Request Checkout Submit (https://developer.niketech.com/docs/projects/Checkouts%20V2?tab=api#checkout-request-a-checkout-submit-put-1) includes 1) a client-generated UUID (checkout ID) in the URL and 2) an Entity in the request body. There are 4 possible scenarios:

Scenario	Result
UUID and Entity are new to the system (base use case)	Client receives HTTP 202 response, request processed as new job
UUID and Entity match a prior request	Client receives the exact same HTTP 202 response from the prior request (no new job processed)
UUID used previously, Entity is new	Client receives HTTP 409 error response (no new job processed)
UUID is new, Entity previously submitted under another UUID	Client receives HTTP 202 response, request processed as new job

TIP: For more, see the Idempotence Guarantee section of the Using Nike APIs (https://developer.niketech.com/nde-docs/doc/getting-started/using-nike-apis.html#idempotence-guarantee) guide.

Conditions for Retries

For retry information by Checkout endpoint, visit Retry Patterns for Checkout Clients (https://confluence.nike.com/display/DAHP/DRAFT+-+Retry+Pattern+for+Checkout+Service+Clients) in Confluence.

For all Checkout APIs, the general rule is that HTTP 4XX error codes (except for 429) should not be retried but HTTP 5XX errors can be retried. For general information on Nike error retry practices, see API Error Patterns (https://confluence.nike.com/pages/

viewpage.action?spaceKey=DAHP&title=API+-+Error+Patterns#API-ErrorPatterns-RetrylogicbasedonHTTPstatuscode) on Confluence.

Honor the ETAs for Best Performance

For async endpoints that return an ETA, i.e. the estimated time for the job to be completed, it is important for your app to honor the ETA for best performance. For example, if the ETA is 2000ms, your app should wait 2000ms to start polling the /jobs endpoint to avoid consuming network and other resources unnecessarily.

Test Scenarios

Here is an example list of test scenarios for a consumer experience that is integrating with the Checkout APIs. Note: in this context, inline refers to a regular Nike product as opposed to a customized Nike iD product.

- Single Product (Inline) - Single Payment (Credit Card)
- Single Product (Inline) - Single Payment (Gift Card)
- Single Product (Inline) - Single Payment (PayPal)
- Single Product (NikeID) - Single Payment (Credit Card)
- Single Product (NikeID) - Single Payment (PayPal)
- Single Product (Inline) - Multiple Payment (Credit Card & Gift Card)
- Single Product (NikeID) - Multiple Payment (Credit Card & Gift Card)
- Mixed Products (Inline & NikeID) - Single Payment (Credit Card)
- Mixed Products (Inline & NikeID) - Single Payment (PayPal)
- Mixed Products (Inline & NikeID) - Multiple Payment (Credit Card & Gift Card)

Additionally, it's useful to add scenarios for multi-quantity (i.e. quantity > 1) for Inline products, as well as scenarios with multiple Nike iD products in same checkout.

TIP: While inspecting browser activity on www.nike.com/launch, you can change your shopping country with the flag icon at the upper right of the homepage. Also, as necessary you can place an order to observe all the checkout calls. Orders can be cancelled via self-service within 30 minutes of submission, otherwise contact Nike Customer Service.

Test Environment

It is recommended to test all Checkout endpoints in the production environment as opposed to the test environment. Using the test environment can have unpredictable results due to the many downstream services which these endpoints are reliant upon in order to provide typical 'production-like' responses.

There are boundaries for testing in production:

- Performance tests at high volumes should never be done in production.
- Calling 'Request a Checkout Submit' in production can result in actual Nike orders being sent for fulfillment, and actual payment methods being authorized and/or charged. Proceed with caution.

Caching Data

None of the endpoints described in this document support caching.

Error Handling: Which JSON Field Had The Error?

In error responses from APIs, Nike uses the **JSON Pointer (https://tools.ietf.org/html/rfc6901)** standard to indicate which field of the request JSON had the error.

However, not all Checkout APIs are the same in this regard. This is due to some APIs having been built before Nike decided to use JSON Pointer standard.

For example, error responses from the Checkouts v2 API will vary from error responses from the Carts API.

Examples of both:

Checkouts (built before adopting JSON Pointer standard):

```
{
    "message": "Validation Failed",
    "errors": [{
        "field": "request.items[0].contactInfo.email",
        "code": "INVALID_EMAIL",
        "message": "Invalid email"
    }]
}
```

Carts (built after adopting JSON Pointer standard):

```
{
    "message": "Validation Failed",
    "errors": [{
        "field": "/request/items/0/contactInfo/email",
        "code": "INVALID_EMAIL",
        "message": "Invalid email"
    }]
}
```

If you are a client of both of the above APIs, you will need to parse error responses in two different ways.

Troubleshooting

Listed below are ways to troubleshoot unexpected responses using this API.

Use Troubleshooting Tools

- Use the general troubleshooting tips in the **Using Nike APIs (https://developer.niketech.com/nde-docs/doc/getting-started/using-nike-apis.html#troubleshooting)** guide.
- Use a Splunk query (requires access) to check for issues with your request.
- Contact the Buy team on the **#cic-order-integration (https://nikedigital.slack.com/messages/C38BE20SV)** Slack channel for assistance.

Common Questions

Is it okay to call Checkout APIs if my app is hosted in an Amazon Web Services VPC?

Yes. The APIs are exposed publicly so it should not matter where you are calling from. If you are calling repeatedly from a small set of IP addresses, it might be possible that Nike's bot-mitigation tools could interfere with your ability to make calls. If you are having issues, reach out to us for help.

My Checkout Submit job is taking a long time to complete. What gives?

Checkout Submits initiate a lot of behind-the-scenes API calls, the duration of which is somewhat unpredictable. Depending on the total volume of requests happening at the time your request was submitted, combined with the payment method and shipping country selected by the consumer, it may take several seconds to get a completed job. Best case is about 5 seconds, worst case can be well over a minute. If the job times out, you will get a 'completed with error' job status.

Terms of Service

It is recommended that you send a caller ID header in every request to this API to help troubleshoot unexpected responses. See the Registration section of the **Using Nike APIs (https://developer.niketech.com/nde-docs/doc/getting-started/using-nike-apis.html#registration)** guide on how to create and register your caller ID.

Authentication

Access Tokens

Most calls through the Nike API gateway (api.nike.com) require an access token be sent in the request header. This allows Nike to verify that your app is authorized to perform the action on behalf of the consumer. Access tokens are obtained by calling Nike Unite services prior to calling the API which you ultimately want to reach.

See **Using Nike APIs (https://developer.niketech.com/nde-docs/doc/getting-started/using-nike-apis.html#authorization)** guide for more on how to call Unite services.

JSON Web Token

Only one endpoint in the Buy APIs, Launch Checkout Submit, requires the additional authorization of a JSON Web Token (JWT). For more, see the JWT section of **Using Nike APIs (https://developer.niketech.com/nde-docs/doc/getting-started/using-nike-apis.html#jwt-json-web-token)**.

Contacting the Team

Need to contact the Cart & Checkout team?

Slack	**#cic-order-integration (https://nikedigital.slack.com/messages/C38BE20SV)**
Confluence Space	**CiC Order Capture (https://confluence.nike.com/pages/viewpage.action?pageId=163654070)**
Team Contacts	**Dan Robertson**, **Saket Shrivastava**, **Sree Krishna** (Carts, Wish Lists, Address Validation)

Document Change Log

Summary	Date
Initial publish	10/02/2018
Added Carts v2 API	03/16/2018
Added Wish Lists API	04/30/2018
Added Key Terms	08/01/2019
Added Address Validation	09/30/2019

Next Steps

You've learned how to add Cart & Checkout to your experience. Here are some next steps.

- **Adding Payment to Your Experience (https://developer.niketech.com/nde-docs/doc/commerce/payment/use-payment.html)**
- **Using Nike APIs (https://developer.niketech.com/nde-docs/doc/getting-started/using-**

nike-apis.html)

- **Glossary (https://developer.niketech.com/nde-docs/doc/commerce/reference/glossary.html)**
- **Supported Countries & Currencies (https://developer.niketech.com/nde-docs/doc/commerce/checkout/checkout-country-currency.html)**